Universite de Rouen Normandie

Master 1 – Genie Informatique et Logiciel

Projet Gestion de Flotte de vehicules

Rapport de Semaine 2

Infrastructure DevOps – Conteneurisation & Orchestration

Etudiant :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Annee universitaire :
2025 – 2026

Semaine :
2 / 7

19 mars 2026

Table des matières

1 Introduction et objectifs de la semaine 2

2 Environnement de developpement local – Docker Compose 2

3 Conteneurisation avec Docker 3
3.1 Concept . . . 3
3.2 Dockerfile multi-stage pour vehicle-service . . . 3
3.3 Construction de l'image dans Minikube . . . 4

4 Orchestration avec Kubernetes (Minikube) 4
4.1 Concept . . . 5
4.2 Isolation du projet – Namespace . . . 5
4.3 Les 4 objets Kubernetes mis en place . . . 5
4.3.1 Deployment – Gestion des conteneurs . . . 5
4.3.2 Service – Adresse reseau stable . . . 5
4.3.3 Secret – Gestion securisee des mots de passe . . . 6
4.3.4 Ingress – Point d'entree externe . . . 6

5 Infrastructure as Code avec Helm 7
5.1 Concept . . . 7
5.2 Strategie de deploiement unifiee . . . 7
5.3 Separation des secrets Helm . . . 8

6 Resolution du probleme Kafka 8
6.1 Probleme rencontre . . . 8
6.2 Pivot technique – Image Confluent officielle . . . 8

7 Pipeline CI/CD – Automatisation avec GitHub Actions 9
7.1 Description des jobs de la CI/CD . . . 9
7.2 Ameliorations futures de la CI/CD . . . 9

8 Observabilite – Stack OpenTelemetry & Supervision 10
8.1 Architecture de la stack . . . 11
8.2 Deploiement Local – Extension via Compose Override . . . 11
8.3 Deploiement Kubernetes – Isolation et Ingress . . . 11

9 Bilan de la semaine 2 12
9.1 Etat final du cluster . . . 12
9.2 Structure finale du projet . . . 12
9.3 Points d'attention pour la suite . . . 13

10 Conclusion 13

1. Introduction et objectifs de la semaine

Objectif de la semaine 2 : Mettre en place l'integralite de l'environnement d'infrastructure avant d'ecrire la moindre ligne de code metier. Cette semaine constitue le socle sur lequel tous les microservices seront deployes dans les semaines suivantes.

La semaine 2 du projet est consacree a la mise en place de l'infrastructure DevOps complete. L'objectif est de disposer, en fin de semaine, d'un environnement reproductible et fonctionnel sur lequel n'importe quel membre de l'equipe peut travailler sans configuration manuelle.

Les grandes etapes realisees sont les suivantes :

1. Conteneurisation des services avec **Docker** (Dockerfiles multi-stage)
2. Orchestration locale avec **Kubernetes** via Minikube
3. Mise en place d'une architecture **Helm unifiee** pour l'integralite du cluster
4. Centralisation de la stack d'observabilite (**OpenTelemetry, Jaeger, Prometheus, Loki, Grafana**)
5. Unification des configurations entre Docker Compose et Kubernetes via Helm

2. Environnement de developpement local – Docker Compose

Principe : Docker Compose sert uniquement au developpement rapide en local. Kubernetes (Minikube) simule l'environnement de production. Les deux sont independants et ne se parlent pas.

A la racine du projet, un fichier `compose.yaml` permet de lancer instantanement toute l'infrastructure necessaire au developpement sans passer par Kubernetes. Il contient 5 services :

Conteneur	Image	Port	Role
flotte-postgres	timescale/timescaledb :2.14.2-pg15	5432	Base de donnees principale + time-serie
flotte-redis	redis :7.2-alpine	6379	Cache et service evenements
flotte-kafka	apache/kafka :3.7.0	9092	Bus de messages (mode KRaft)
flotte-keycloak	keycloak :24.0.2	8180	Authentification SSO
flotte-pgadmin	dpage/pgadmin4	5050	Interface web PostgreSQL

Table 1 – Services du Docker Compose de developpement local

Choix techniques notables :

— **TimescaleDB** a la place de PostgreSQL standard – supporte nativement les donnees time-series pour le service de localisation GPS

— **Kafka en mode KRaft** – sans Zookeeper, architecture simplifiee

— **Keycloak** integre des le developpement local pour tester l'authentification SSO sans attendre Kubernetes

> **Note :** Deux images Kafka differentes sont utilisees selon l'environnement : `apache/kafka:3.7.0` en developpement local (Docker Compose) et `confluentinc/cp-kafka:7.5.0` sur Kubernetes. Ce choix est volontaire – l'image Apache etant disponible sur Docker Hub pour le dev local, et l'image Confluent etant plus stable pour l'orchestration Kubernetes. Le protocole Kafka etant standardise, les deux sont entierement compatibles.

Le lancement se fait en une seule commande :

```
docker compose up -d
```

Listing 1 – Lancement de l'environnement de developpement local

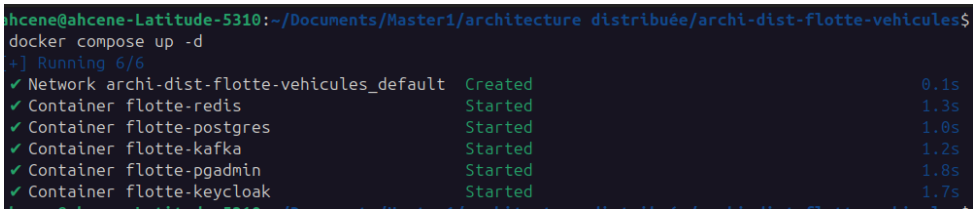

FIGURE 1 – Les 5 conteneurs du Docker Compose demarres et operationnels

pgAdmin est accessible sur `http://localhost:5050` et Keycloak sur `http://localhost:8080`. Ces interfaces permettent de visualiser et administrer les donnees sans ligne de commande pendant le developpement.

3. Conteneurisation avec Docker

3.1. Concept

Docker permet d'empaqueter une application et toutes ses dependances dans une image standardisee, garantissant qu'elle fonctionnera de maniere identique sur n'importe quel environnement (poste de developpement, CI/CD, production).

3.2. Dockerfile multi-stage pour vehicle-service

Le choix d'un **Dockerfile multi-stage** permet de separer la phase de compilation de la phase d'execution, reduisant ainsi la taille de l'image finale et ameliorant la securite en n'incluant pas les outils de build en production.

```
# Etape 1 : Construction
FROM maven:3.9.6-eclipse-temurin-21-alpine AS builder

WORKDIR /build

# On copie d'abord le pom.xml pour telecharger les dependances
COPY pom.xml .
```

```
RUN mvn dependency:go-offline

# On copie tout le code source et on compile le projet
COPY src ./src
RUN mvn clean package -DskipTests

# Etape 2 : Production
FROM eclipse-temurin:21-jre-alpine

WORKDIR /app

# Securite : creation d'un utilisateur non-root
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

# On recupere l'application compilee depuis l'etape 1
COPY --from=builder /build/target/*.jar app.jar

# On expose le port de Spring Boot
EXPOSE 8080

# Commande de lancement
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 2 – Dockerfile multi-stage - vehicle-service (Spring Boot)

Points cles du Dockerfile :

- `imagePullPolicy: Never` – obligatoire en local pour utiliser les images construites directement dans l'environnement Docker de Minikube
- Utilisation d'un utilisateur non-root pour la securite
- Separation build / production pour reduire la taille de l'image finale

3.3. Construction de l'image dans Minikube

Pour eviter d'avoir a pousser les images sur un registre distant (Docker Hub), les images ont ete construites directement dans l'environnement Docker interne de Minikube :

```
# Pointer le daemon Docker vers Minikube
eval $(minikube docker-env)

# Builder l'image directement dans Minikube
docker build -t vehicle-service:latest ./services/vehicle-service/

# Verifier que l'image est disponible
minikube image ls | grep vehicle
```

Listing 3 – Construction de l'image dans Minikube

4. Orchestration avec Kubernetes (Minikube)

```
ahcene@ahcene-Latitude-5310:~$ minikube image ls | grep vehicle
docker.io/library/vehicle-service:latest
```

FIGURE 2 – Construction de l'image vehicle-service dans Minikube

4.1. Concept

Kubernetes est le chef d'orchestre des conteneurs. Il garantit que les services restent disponibles (redemarrage automatique en cas de crash), gere le reseau interne entre services, et permet la mise a l'echelle. **Minikube** simule un cluster Kubernetes professionnel sur une seule machine locale.

4.2. Isolation du projet – Namespace

Un **Namespace** a ete cree pour isoler l'integralite du projet dans un espace dedie, separe du reste du systeme Kubernetes :

```
kubectl apply -f infra/kubernetes/namespaces/namespace.yaml
kubectl get namespaces
```

Listing 4 – Creation du namespace

4.3. Les 4 objets Kubernetes mis en place

4.3.1. Deployment – Gestion des conteneurs

Le **Deployment** declare quoi faire tourner, combien de replicas maintenir, et relance automatiquement les pods en cas de crash. Un Deployment a ete cree pour chacun des 5 microservices avec 2 replicas chacun.

```
vehicle-deployment-75c88b967b-8vfb9      1/1     Running            1 (15h ago)   21h
vehicle-deployment-75c88b967b-mjxcq      1/1     Running            1 (15h ago)   21h
```

FIGURE 3 – Exemple de Deployment Kubernetes – vehicle-service

4.3.2. Service – Adresse reseau stable

Les pods etant ephemeres (leur IP change a chaque redemarrage), le **Service** Kubernetes fournit une adresse reseau stable et permanente. Tous les services sont de type `ClusterIP` (accessibles uniquement en interne au cluster).

Service	Technologie	Port	Type
vehicle-service	Spring Boot	8080	ClusterIP
driver-service	Spring Boot	8081	ClusterIP
maintenance-service	Spring Boot	8082	ClusterIP
event-service	Spring Boot	8083	ClusterIP
location-service	Node.js	3000	ClusterIP

TABLE 2 – Recapitulatif des Services Kubernetes deployes

4.3.3. Secret – Gestion securisee des mots de passe

Les mots de passe et URLs de connexion ne sont jamais ecrits en dur dans le code source ni dans les Deployments. Un **Secret Kubernetes** centralise ces donnees sensibles et les injecte comme variables d'environnement dans les pods au demarrage.

Dans le fichier `application.yaml` de Spring Boot, la syntaxe `${VARIABLE:valeur_defaut}` permet d'utiliser la variable injectee par le Secret, avec une valeur de fallback pour le developpement local :

```
spring:
  datasource:
    url: ${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/
       flotte_db}
    username: ${SPRING_DATASOURCE_USERNAME:admin}
    password: ${SPRING_DATASOURCE_PASSWORD:password}
  kafka:
    bootstrap-servers: ${KAFKA_BROKER:localhost:9092}
```

Listing 5 – application.yaml – injection des secrets via variables d'environnement

Le fichier `db-secrets.yaml` est ajoute au `.gitignore` pour ne jamais etre commite sur Git. Un fichier `values.secret.yaml.example` sert de template pour les membres de l'equipe.

4.3.4. Ingress – Point d'entree externe

L'**Ingress** expose le cluster vers l'exterieur et route les requetes HTTP vers le bon service selon le chemin URL. Le controller NGINX a ete active via l'addon Minikube :

```
# Activer l'addon Ingress NGINX
minikube addons enable ingress

# Ajouter fleet.local dans /etc/hosts
echo "$(minikube ip) flotte.local" | sudo tee -a /etc/hosts

# Appliquer le manifest
kubectl apply -f infra/kubernetes/ingress/ingress.yaml
```

Listing 6 – Activation et configuration de l'Ingress

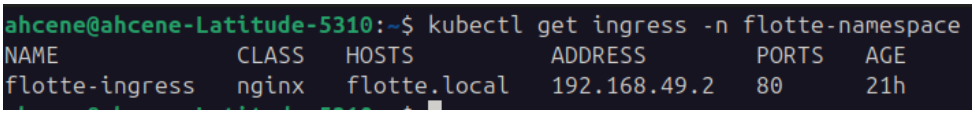

FIGURE 4 – Ingress deploye avec l'adresse IP de Minikube

La table de routage configuree est la suivante :

URL	Service cible
`fleet.local/api/vehicles`	vehicle-service :8080
`fleet.local/api/drivers`	driver-service :8081
`fleet.local/api/maintenance`	maintenance-service :8082
`fleet.local/api/events`	event-service :8083
`fleet.local/api/location`	location-service :3000

Table 3 – Table de routage de l'Ingress

5. Infrastructure as Code avec Helm

5.1. Concept

Helm est le gestionnaire de paquets de Kubernetes. Plutot que d'ecrire et maintenir manuellement des dizaines de fichiers YAML pour des services tiers complexes, Helm utilise des **Charts** – des packages preconfigures et maintenus par la communaute.

5.2. Strategie de deploiement unifiee

Le projet a migre vers une architecture **Full Helm**. Plutot que de separer les manifests des microservices de la configuration de l'infrastructure, nous avons cree trois Charts principaux :

— **Infrastructure de base** (`fleet-infra`) : PostgreSQL, Redis via Bitnami, et Kafka (manifest KRaft).

— **Stack d'observabilite** (`fleet-observability`) : Toute la stack LGTMe (Loki, Grafana, Tempo, Prometheus) et l'OTel Collector.

— **Application** (`fleet-app`) : Contient uniquement nos microservices metiers et les regles d'Ingress.

Cette separation en trois entites distinctes permet une gestion independante du cycle de vie des bases de donnees, du monitoring et du code metier. Les fichiers de configuration (Prometheus, OTel Collector) sont centralises dans le chart d'observabilite et partages entre Docker Compose (via volumes) et Kubernetes (via ConfigMaps generes par Helm).

Cela permet a n'importe quel developpeur de lancer toute l'infrastructure en **3 commandes seulement** :

```
# 1. Installer l'infra de base (DBs, Kafka)
helm upgrade --install fleet-infra ./infra/helm/fleet-infra/ -n
    flotte-namespace
# 2. Installer la stack d'observabilite (LGTMe + OTel)
helm upgrade --install fleet-obs ./infra/helm/fleet-observability/
    -n flotte-namespace
# 3. Installer les microservices applicatifs
helm upgrade --install fleet-app ./infra/helm/fleet-app/ -n flotte-
    namespace
```

Listing 7 – Lancement complet de l'infrastructure

5.3. Separation des secrets Helm

Pour ne pas exposer les mots de passe sur Git, une approche a deux fichiers a ete adoptee :

- `values.yaml` – configuration non-sensible, commite sur Git
- `values.secret.yaml` – mots de passe uniquement, dans `.gitignore`
- `values.secret.yaml.example` – template pour les coeequipiers, commite sur Git

```
postgres-service-0                    1/1     Running            1 (15h ago)   22h
redis-service-master-0                1/1     Running            1 (15h ago)   22h
```

FIGURE 5 – PostgreSQL et Redis deployes et Running via Helm Bitnami

6. Resolution du probleme Kafka

6.1. Probleme rencontre

Le deploiement de Kafka via le Chart Helm Bitnami a echoue avec l'erreur `Init:ImagePullBackOff`. L'investigation via `kubectl describe pod` a revele la cause exacte :

```
Warning Failed  kubelet  Failed to pull image "bitnami/kafka:3.7.0"
    :
Error response from daemon: manifest unknown: manifest unknown
```

Listing 8 – Message d'erreur lors du describe pod

Cause identifiee : Bitnami a retire ses images de Docker Hub depuis 2023. Les tags versionnels (`bitnami/kafka:3.7.0`, `latest`) n'existent plus sur les registres publics accessibles depuis Minikube.

6.2. Pivot technique – Image Confluent officielle

Plutot que de s'acharner sur un outil instable, la decision a ete prise d'abandonner le Chart Bitnami pour Kafka et d'utiliser un manifest Kubernetes classique avec l'image officielle **Confluent Platform** (`confluentinc/cp-kafka:7.5.0`), equivalent a Apache Kafka 3.5.x.

Avantages de cette approche :

- Configuration en mode **KRaft** (sans Zookeeper) – architecture simplifiee
- Image Confluent, reference industrielle utilisee en production
- Controle total de la configuration via variables d'environnement
- Demarrage instantane, pas d'init containers problematiques

```
# Deploiement via manifest classique avec image Confluent
kubectl apply -f infra/kubernetes/kafka/kafka.yaml

# Verification que Kafka tourne
kubectl get pods -n flotte-namespace | grep kafka
```

Listing 9 – Solution – deploiement Kafka via manifest Kubernetes

```
kafka-deployment-757f75f779-wdjnr          1/1     Running             1 (15h ago)   22h
```

FIGURE 6 – Kafka deploye et Running via manifest Kubernetes – image confluentinc/cp-kafka :7.5.0 (Kafka 3.5)

7. Pipeline CI/CD – Automatisation avec GitHub Actions

> **Principe :** Le pipeline CI/CD assure la qualite du code et la validite de l'infrastructure a chaque modification. Il automatise les tests et le deploiement vers le registre d'images GitHub (GHCR).

Le projet utilise **GitHub Actions** pour orchestrer le cycle de vie du developpement. Le workflow est defini dans `.github/workflows/ci-cd.yml`.

7.1. Description des jobs de la CI/CD

Le pipeline se decompose en quatre jobs principaux :

1. **Tests des microservices Java (Spring Boot) :**
   - Utilisation d'une **matrice de build** pour tester en parallele les services : `driver`, `events`, `maintenance` et `vehicle`.
   - **Utilisation des services natifs GitHub :** des instances **PostgreSQL 15** et **Kafka 3.7.0** sont lancees dynamiquement pour chaque job afin de valider le contexte Spring Boot lors des tests d'integration.
2. **Tests du microservice Node.js :**
   - Pipeline dedie pour le `location-service` utilisant Node.js 20.
   - Installation propre via `npm ci` et execution des tests unitaires.
3. **Validation de l'infrastructure (Linting) :**
   - Execution de `helm lint` sur les charts de dependances pour garantir la validite syntaxique des manifests Kubernetes generes.
4. **Build & Push Docker (CD) :**
   - Ce job ne s'execute que si les trois etapes precedentes sont en succes et uniquement lors d'un push sur la branche `main`.
   - Utilisation de **Docker Buildx** pour optimiser le cache de construction.
   - Publication sur **GitHub Container Registry (GHCR)** avec un double marquage : `:latest` et le `SHA` du commit pour la tracabilite.

7.2. Ameliorations futures de la CI/CD

L'infrastructure actuelle se concentre sur la validation fonctionnelle. Pour la suite du projet, nous prevoyons de renforcer la qualite et la securite des livrables en ajoutant des etapes de controle supplementaires :

— **Conformite des Dockerfiles :** Mise en place de **Hadolint** pour verifier que les Dockerfiles respectent les bonnes pratiques (optimisation des couches, suppression des caches, etc.).

— **Securite des images (Vulnerability Scanning) :** Integration de **Trivy** ou **Snyk** pour detecter les vulnerabilites CVE dans les images de base et les dependances installees.

— **Analyse statique de code (SAST) :** Ajout de **SonarCloud** ou **CodeQL** pour identifier les bugs potentiels et les failles de securite directement dans le code source Java et JavaScript.

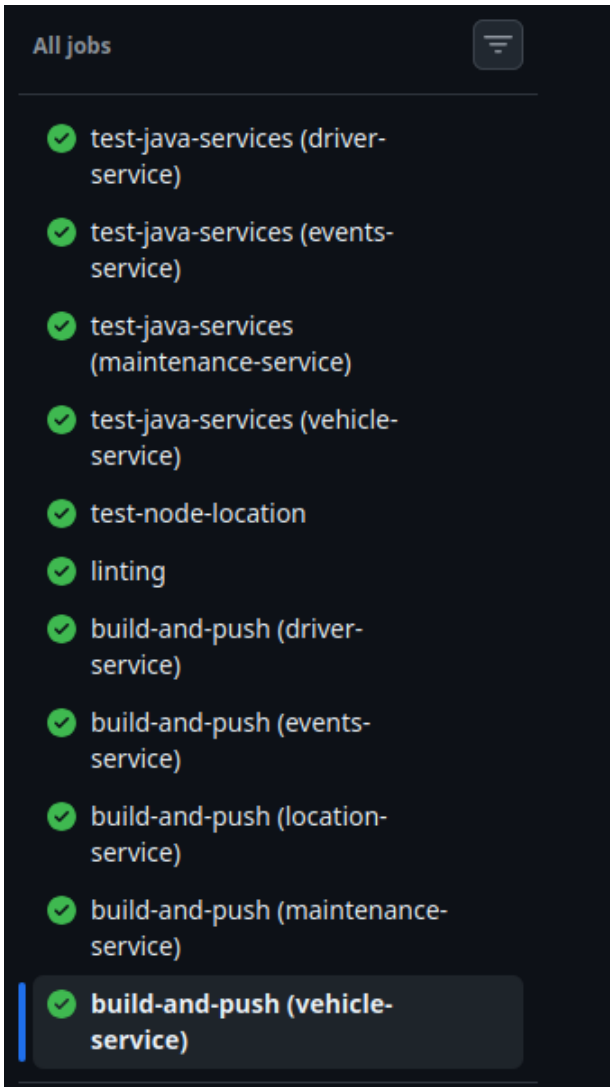

FIGURE 7 – Visualisation symbolique de l'infrastructure validee par le pipeline

8. Observabilite – Stack OpenTelemetry & Supervision

Principe : L'observabilite repose sur les trois piliers : **Traces**, **Metriques** et **Logs**. Nous utilisons **OpenTelemetry (OTel)** comme standard d'ingestion unique pour decorreler la generation des donnees de leur stockage final.

8.1. Architecture de la stack

La stack d'observabilite est composee de 5 outils majeurs :

- **OpenTelemetry Collector :** Point d'entree unique qui recoit, traite et exporte les donnees vers les differents backends.
- **Jaeger :** Stockage et visualisation des traces distribuees (tracing).
- **Prometheus :** Stockage des metriques de performance.
- **Loki :** Agregation et stockage des journaux (logs).
- **Grafana :** Interface de visualisation unifiee pour les trois types de donnees.

8.2. Deploiement Local – Extension via Compose Override

Pour ne pas surcharger le `compose.yaml` principal tout en facilitant le developpement, un fichier `compose.override.yaml` a ete cree. Docker Compose fusionne automatiquement ce fichier avec la configuration de base lors du lancement. Cela permet de disposer de la stack d'observabilite par defaut sans argument supplementaire :

```
# Docker detecte et fusionne automatiquement compose.yaml et compose.override.yaml
docker compose up -d
```

Listing 10 – Lancement de l'integralite de l'environnement (Infrastructure + Logs)

Les fichiers de configuration (OTel Collector et Prometheus) sont centralises dans le dossier `infra/helm/fleet-observability/config/`. Ces fichiers servent a la fois pour les montages de volume Docker Compose et pour les ConfigMaps Kubernetes via la fonction `.Files.Get` de Helm.

8.3. Deploiement Kubernetes – Isolation et Ingress

Sur Kubernetes, chaque composant est deploye dans le namespace `flotte-namespace` avec ses propres `Deployment` et `Service`.

Acces simplifie via l'Ingress : Afin d'eviter l'utilisation de `port-forward`, des regles d'Ingress ont ete ajoutees pour acceder aux interfaces graphiques via le domaine `flotte.local` :

URL	Interface
`flotte.local/grafana`	Tableaux de bord (admin/admin)
`flotte.local/jaeger`	Tracing des requetes

Table 4 – Acces externes aux outils d'observabilite

Une configuration specifique (`root_url` pour Grafana et `base-path` pour Jaeger) a ete appliquee pour permettre le fonctionnement correct derriere ces sous-chemins URLs.

9. Bilan de la semaine 2

9.1. Etat final du cluster

Voici l'etat du cluster pour le moment :

```
ahcene@ahcene-Latitude-5310:~/Documents/Master1/architecture distribuée/archi-dist-flotte-vehicules$ kubectl get pods -n flotte-namespace -w
NAME                                    READY   STATUS             RESTARTS      AGE
driver-deployment-85784ddb97-2rn6k      0/1     ErrImageNeverPull  0             21h
driver-deployment-85784ddb97-j2ffj      0/1     ErrImageNeverPull  0             21h
event-deployment-847858d845-ks5cr       0/1     ErrImageNeverPull  0             21h
event-deployment-847858d845-n4tf9       0/1     ErrImageNeverPull  0             21h
kafka-deployment-757f75f779-wdjnr       1/1     Running            1 (15h ago)   22h
location-deployment-5589c88f48-ctqr5    0/1     ErrImageNeverPull  0             21h
location-deployment-5589c88f48-m2ht7    0/1     ErrImageNeverPull  0             21h
maintenance-deployment-6f87b585df-n5ckq 0/1     ErrImageNeverPull  0             21h
maintenance-deployment-6f87b585df-svx2q 0/1     ErrImageNeverPull  0             21h
postgres-service-0                      1/1     Running            1 (15h ago)   22h
redis-service-master-0                  1/1     Running            1 (15h ago)   22h
vehicle-deployment-75c88b967b-8vfb9     1/1     Running            1 (15h ago)   21h
vehicle-deployment-75c88b967b-mjxcq     1/1     Running            1 (15h ago)   21h
```

FIGURE 8 – Etat final du cluster en fin de semaine 2

Composant	Statut	Methode de deploiement
PostgreSQL	Running	Helm (`fleet-infra`)
Redis	Running	Helm (`fleet-infra`)
Kafka	Running	Helm (`fleet-infra`)
OTel Collector	Running	Helm (`fleet-observability`)
Jaeger / Prometheus	Running	Helm (`fleet-observability`)
Grafana / Loki	Running	Helm (`fleet-observability`)
vehicle-service	Running	Helm (`fleet-app`)
Autres services	Pending	Helm (`fleet-app`)

TABLE 5 – Bilan de l'etat des composants unifies sous Helm

> L'etat `ErrImageNeverPull` des services non encore developpes est **normal et attendu** – les images Docker n'existent pas encore car le code metier sera ecrit lors des semaines 3 a 5. La structure Kubernetes est en place et prete a les accueillir.

9.2. Structure finale du projet

```
racine-du-projet/
|-- compose.yaml                    # Env. de dev (Postgres, Kafka, Keycloak)
|-- compose.override.yaml           # Stack LGTMe (montage volumes config Helm)
|-- infra/
|    |-- helm/
|    |    |-- fleet-infra/          # Chart INFRA (DBs, Kafka)
|    |    |    |-- values.yaml      # Config bases de donnees et bus
```

```
|   |   |   `-- templates/          # Kafka, Postgres (si custom)
|   |   |-- fleet-observability/    # Chart OBSERVABILITE (LGTMe + OTel)
|   |   |   |-- values.yaml         # Config logs, traces et metriques
|   |   |   `-- config/             # SOURCE DE VERITE UNIQUE (OTel, Prom)
|   |   `-- fleet-app/              # Chart APPLICATIF (Microservices)
|   |       |-- values.yaml         # Config des services metiers
|   |       `-- templates/          # Microservices, Ingress
|   `-- kubernetes/
|       |-- namespaces/
|       `-- secrets/
```

Listing 11 – Arborescence des dossiers d'infrastructure

9.3. Points d'attention pour la suite

- **Semaine 3 :** Coder le vehicle-service (CRUD REST, Kafka producer/consumer)
- **Instrumentation :** Integrer les SDK OpenTelemetry dans les microservices pour envoyer les traces et metriques au Collector.
- **Tableaux de bord :** Creer les dashboards Grafana de reference pour le monitoring des microservices.
- **Semaine 7 :** Packager les microservices dans un Chart Helm pour le deploiement final prod-ready

10. Conclusion

La semaine 2 a permis de construire un environnement d'infrastructure solide et reproductible. Le principal defi rencontre – l'instabilite du Chart Helm Kafka de Bitnami – a ete resolu par un **pivot technique rapide** vers un manifest Kubernetes classique utilisant l'image officielle Apache Kafka. Cette decision illustre l'importance de l'agilite dans un projet d'architecture distribuee : identifier rapidement qu'un outil pose probleme et le remplacer par une solution plus simple et plus fiable.

En fin de semaine, l'ensemble des composants d'infrastructure (PostgreSQL, Redis, Kafka, Ingress, Secrets) est operationnel dans le cluster Minikube. Le service `vehicle-service` tourne deja en `Running`, validant le bon fonctionnement de la chaine complete : Dockerfile → image Minikube → Deployment → Service → Ingress.